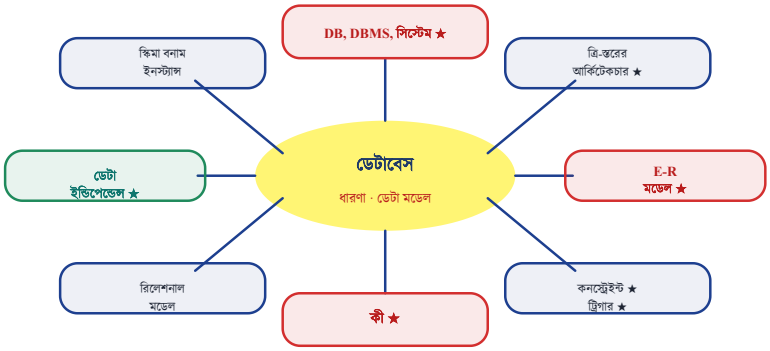
ডেটাবেস ধারণা ও ডেটা মডেল

DB · DBMS · ত্রি স্তর · E-R · রিলেশনাল · কী · কনস্ট্রেইন্ট · ট্রিগার



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = গোটা আর্কিটেকচার যার জন্য তৈরি

সিলেবাসের যে অংশ

ডেটাবেস সিস্টেমের সিলেবাস লেখা আছে: "Concepts of database systems; Data Models: **Entity-Relationship model, Relational model**; Query Languages: Relational algebra, SQL; **Constraints and triggers**; Functional dependencies and normalization; File organization and data storage; Indexing; Query optimization; Transaction management; Recovery; Concurrency control; Access control and security; Semi-structured database; Object oriented databases." **এই লেসন কেবল প্রথম তিনটি অংশ নেয়।**

হাই-ইল্ড তালিকা

① ডেটাবেস সিস্টেম = **DB + DBMS + অ্যাপ + ব্যবহারকারী**
② স্কিমা = ব্লুপ্রিন্ট · ইনস্ট্যান্স = এখনকার ডেটা
③ এক্সটার্নাল → কনসেপচুয়াল → ইন্টার্নাল
④ ইন্ডিপেন্ডেন্স: **লজিক্যাল**, **ফিজিক্যাল**
⑤ চেন: আয়তক্ষেত্র = এন্টিটি, ডিম্ব = অ্যাট্রিবিউট, লম্ব = সম্পর্ক, আন্ডারলাইন = কী
⑥ ওয়াক এন্টিটি = **ডাবল রেক্টাঙ্গল**
⑦ ১:N ⇒ "এক" পাশের PK, "অনেক" পাশে FK
⑧ M:N ⇒ **নিজস্ব জাংশন টেবিল লাগেই**
⑨ FK ⇒ **রেফারেনশিয়াল**, PK ⇒ **এনটিটি** ইন্টেগ্রিটি
⑩ **DROP** কাঠামো মুছে, DELETE/TRUNCATE মোছে না

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = অবশ্যই মুখস্থ · **বেগুনি** = মনে রাখার কৌশল · **লাল** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ ডেটাবেস, DBMS, সিস্টেম … পৃ. ২
✓ ত্রি স্তর; স্কিমা বনাম ইনস্ট্যান্স … পৃ. ৩
✓ ডেটা ইন্ডিপেন্ডেন্স … পৃ. ৪
✓ E-R মডেল, চেনের নোটেশন … পৃ. ৫
✓ কার্ডিনালিটি ১:১, ১:N, M:N … পৃ. ৬
✓ রিলেশনাল মডেল … পৃ. ৭
✓ কী … পৃ. ৮
✓ কনস্ট্রেইন্ট ও ইন্টেগ্রিটি … পৃ. ৯
✓ ট্রিগার; E-R বনাম রিলেশনাল … পৃ. ১০
✓ রাতের পরীক্ষার পাতা … পৃ. ১১
✓ অনুশীলনী MCQ + লিখিত … পৃ. ১২

১ ডেটাবেস, DBMS ও ডেটাবেস সিস্টেম

সংজ্ঞা: **ডেটাবেস** হলো সম্পর্কিত ডেটার সাজানো সংগ্রহ। **DBMS** (ডেটাবেস ম্যানেজমেন্ট সিস্টেম) হলো সেই সফটওয়্যার যা ডেটা সংরক্ষণ, পুনরুদ্ধার ও সুরক্ষা করে। আর **ডেটাবেস সিস্টেম** হলো পুরো সমষ্টি: **ডেটাবেস** + **DBMS** + **অ্যাপ্লিকেশন প্রোগ্রাম** + **ব্যবহারকারী** + **হার্ডওয়্যার**। "কোনটি কী?" ধরনের প্রতিটি প্রশ্নের উত্তর নির্ভর করে এই পাঁচটি অংশের কোনটি বোঝানো হচ্ছে।



তিনটি শব্দ, এক স্তূপ

DB = ডেটা, DBMS = সফটওয়্যার

পরিভাষা	আসলে কী	উদাহরণ
ডেটাবেস	নিজেই ডেটা, সাজানো স্থায়ী রূপে	বিশ্ববিদ্যালয়ের ছাত্র, কোর্স, পরীক্ষা
DBMS	অ্যাপ ও ডেটার মাঝের সফটওয়্যার	MySQL, Oracle, PostgreSQL; MongoDB *NoSQL*
ডেটাবেস সিস্টেম	**পাঁচটি অংশ একসঙ্গে**	বিশ্ববিদ্যালয় যা বসিয়ে চালায়
অ্যাপ্লিকেশন প্রোগ্রাম	DBMS-এর কাছে ডেটা চাওয়া প্রোগ্রাম	নম্বর লিপিবদ্ধকারী, ATM, ওয়েবসাইট
হার্ডওয়্যার	ফাইল ও ব্লক ধরে রাখার মেমোরি ও ডিস্ক	ডেটা সত্যিই যেখানে বসে থাকে
ফাইল সিস্টেম	ওএসের ফাইল: নিয়ম নেই, ভাগাভাগি নেই	ডেটাবেস যা প্রতিস্থাপন করেছে
ফাইল সিস্টেম যা নেই	ট্রানজ্যাকশন, কনকারেন্সি নিয়ন্ত্রণ, সুরক্ষা	এখানেই DBMS-এর প্রয়োজন

ডেটাবেস ফাইল কেন হারাল: চারটি ব্যর্থতা, চারটি উত্তর: **অতিরিক্ততা** (এক তথ্য কয়েক ফাইলে) → একটি স্কিমা; **অসামঞ্জস্য** (দুই কপি অমিল) → **কনস্ট্রেইন্ট**; **কম ভাগাভাগি** → ভিউ ও কোয়েরি ভাষা; **প্রোগ্রাম-ডেটা নির্ভরশীলতা** → স্টোরেজের মালিক DBMS।

মনে রাখার কৌশল: **"ডেটা, সফটওয়্যার, সবকিছু।"** DBMS কি ডেটাবেস? **না** — DBMS হলো ম্যানেজার। ডেটাবেস কি DBMS? তাও না। পুরো সমষ্টি বোঝায় শুধু **ডেটাবেস সিস্টেম**।

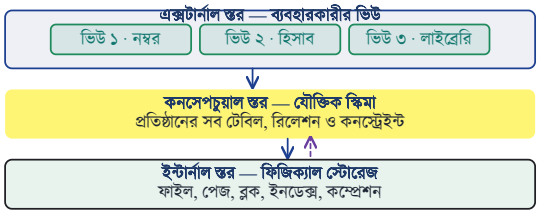
পরীক্ষার ফাঁদ: **ডেটাবেস একটি টেবিল নয়**, আর **ফাইল সিস্টেম ডেটাবেস নয়**: ফাইলে স্কিমা, ট্রানজ্যাকশন বা একাধিক ব্যবহারকারীর নিয়ন্ত্রণ থাকে না। "কোনটি ইন্টেগ্রিটি রক্ষা করে?" → **DBMS**।

সমাধানকৃত উদাহরণ ✍️ একটি ATM: **ডেটাবেস** হলো অ্যাকাউন্ট টেবিল। **DBMS** পিন ডিক্রিপ্ট করে, সারি লক করে যাতে একসঙ্গে দুইজন টাকা না তুলতে পারে, ব্যালেন্স যাচাই করে ও নতুন ব্যালেন্স লেখে। **অ্যাপ্লিকেশন** হলো পর্দার মেনু; **ডেটাবেস সিস্টেম** হলো ব্যাংকের সফটওয়্যার + সার্ভার + কার্ড মেশিন + নেটওয়ার্ক + গ্রাহক।

২ ত্রি-স্তরের আর্কিটেকচার

সংজ্ঞা: **ত্রি-স্তরের (ত্রি-স্কিমা) আর্কিটেকচার**, **ANSI/SPARC**-এর প্রস্তাব, ডেটাবেসকে ভাগ করে **এক্সটার্নাল** স্তরে (ব্যবহারকারী যা দেখে), **কনসেপচুয়াল** স্তরে (পুরো প্রতিষ্ঠানের ডেটার যৌক্তিক কাঠামো) এবং **ইন্টার্নাল** স্তরে (ফিজিক্যালভাবে কীভাবে রাখা হয়)। এদের মাঝে থাকে **ম্যাপিং**, আর ওই ম্যাপিংই **ডেটা ইন্ডিপেন্ডেন্স** তৈরি করে।

ANSI / SPARC



প্রতিটি তির একটি ম্যাপিং, বদলানো যায়

স্তর	কোন প্রশ্নের উত্তর	কে দেখে
এক্সটার্নাল	"এই ব্যবহারকারী কী দেখে?"	একজন ব্যবহারকারী; ভিউ অনেক
কনসেপচুয়াল	"কোন ডেটা আছে, তার অর্থ কী?"	পুরো প্রতিষ্ঠানের যৌক্তিক চোখ
ইন্টার্নাল	"কোথায় ও কীভাবে রাখা?"	শুধু DBMS
ম্যাপিং	"উপরের স্তরের ভাষা নিচেরটায় কীভাবে বাগ করব?"	DBMS-ই লেখে, ব্যবহারকারী নয়

সমাধানকৃত উদাহরণ ✍️ একই টেবিলে স্কিমা বনাম ইনস্ট্যান্স:
স্কিমা হলো `Student(id, name, dept)` — ব্লুপ্রিন্ট, খুব কম বদলায়, থাকে **ডেটা ডিকশনারিতে**। **ইনস্ট্যান্স** হলো `(১০১, "আলিস", "CSE")` — এই মুহূর্তের মান। **স্কিমা আস্তে বদলায়, ইনস্ট্যান্স বারবার।**

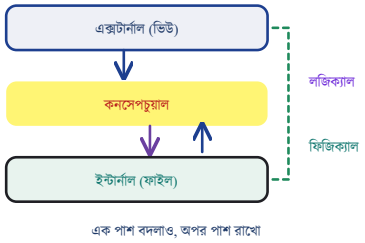
বিষয়	স্কিমা	ইনস্ট্যান্স
আরও নাম	যৌক্তিক নকশা, মেটাডেটা	স্ন্যাপশট, বর্তমান অবস্থা
এতে থাকে	নাম, কলাম, টাইপ, কনস্ট্রেইন্ট	প্রকৃত সংরক্ষিত সারি
বদলায়	খুব কম, ডিজাইনার করে (DDL)	প্রতি মুহূর্তে (DML)
থাকে	ডেটা ডিকশনারিতে	ডেটা ফাইলে
কে দেখে	ডিজাইনার, DBA	ব্যবহারকারী ও অ্যাপ
উদাহরণ	DDL দিয়ে CREATE / ALTER	DML দিয়ে INSERT / UPDATE

মনে রাখার কৌশল: **"বাইরে, ধারণা, ভিতরে।"** *E* = **E**very **u**ser — প্রত্যেক ব্যবহারকারী, *C* = **C**ommunity — সব ডেটার একসঙ্গে, *I* = **I**nternal — ভেতরের পাইপিং। স্তূপ **উপরে নিচে** পড়ো।

পরীক্ষার ফাঁদ: **এক্সটার্নাল স্তর একটি ভিউ নয়** — সাধারণত অনেকগুলো। **ইন্টার্নাল স্তর শুধু DBMS-এর**। **কনসেপচুয়াল স্তর যৌক্তিক**: এতে ফাইলের নাম বা ব্লক সাইজ থাকে না।

৩ তিন স্তর কেন? ডেটা ইন্ডিপেন্ডেন্স

সংজ্ঞা: **ডেটা ইন্ডিপেন্ডেন্স** হলো উপরের স্তর না বদলে কোনো একটি স্তরের স্কিমা বদলানোর সক্ষমতা। এর ধরন মাত্র **দুটি**; "তিন স্তরের ইন্ডিপেন্ডেন্স" বলা একটি প্রশ্ন এখানেই ফাঁদ। **লজিক্যাল** = কনসেপচুয়াল → এক্সটার্নাল; **ফিজিক্যাল** = ইন্টার্নাল → কনসেপচুয়াল।



দুটি তির, দুই ধরনের স্বাধীনতা

দুটি ধরন, ঠিকভাবে: **লজিক্যাল ডেটা ইন্ডিপেন্ডেন্স** — **কনসেপচুয়াল স্কিমা** বদলাও (কলাম যোগ, টেবিল ভাগ) আর **এক্সটার্নাল ভিউ ও প্রোগ্রাম অপরিবর্তিত থাকে**। কঠিন, কারণ ডেটার অর্থই বদলায়।

ফিজিক্যাল ডেটা ইন্ডিপেন্ডেন্স — **ইন্টার্নাল স্কিমা** বদলাও (ইনডেক্স যোগ, ফাইল ভাগ) আর **কনসেপচুয়াল স্কিমা ও প্রোগ্রাম অপরিবর্তিত থাকে**। সহজ, কারণ অর্থ বদলায় না। দুটিই দেওয়া হয় দুটি ম্যাপিং দিয়ে: **এক্সটার্নাল/কনসেপচুয়াল** ও **কনসেপচুয়াল/ইন্টার্নাল**।

	লজিক্যাল ডেটা ইন্ডিপেন্ডেন্স	ফিজিক্যাল ডেটা ইন্ডিপেন্ডেন্স
কোনদের মধ্যে	কনসেপচুয়াল ও এক্সটার্নাল স্তর	ইন্টার্নাল ও কনসেপচুয়াল স্তর
আপনি বদলাই	**যৌক্তিক** স্কিমা: phone যোগ	**স্টোরেজ**: B+ ট্রি ইনডেক্স যোগ
অপ্রভাবিত থাকে	এক্সটার্নাল ভিউ ও অ্যাপ প্রোগ্রাম	কনসেপচুয়াল স্কিমা ও অ্যাপ প্রোগ্রাম
কীসের জন্য	এক্সটার্নাল/কনসেপচুয়াল **ম্যাপিং** লেখা হয়	কনসেপচুয়াল/ইন্টার্নাল **ম্যাপিং** লেখা হয়
সাধারণ উদাহরণ	কোনো কোয়েরি না ছুঁয়েই কলাম যোগ	ইনডেক্স সরালে গতি বদলায়, অর্থ নয়
কতটা কঠিন	**কঠিন** — অর্থ বদলায়	**সহজ** — শুধু বিন্যাস বদলায়
উদাহরণ হিসেবে	এক টেবিল ভেঙে দুই টেবিল, নতুন টেবিল যোগ	নতুন ডিস্ক, পেজ বিন্যাস, কম্প্রেশন
ঝুঁকি	ভুল অর্থ ভাগ করলে ভুল উত্তর	কোয়েরি একই থাকে
এড়িয়ে যায়	কখনো না — ব্যবহারকারীদের সুবিধাই মূল কারণ	প্রায় সবসময়

মনে রাখার কৌশল: "**লজিক্যাল উপরে, ফিজিক্যাল নিচে।**" লজিক্যাল স্বাধীনতা *উপরের* স্তর বদলায়, ফিজিক্যাল *নিচের*।

পরীক্ষার ফাঁদ: ধরন **দুটি**, তিনটি নয়। ইনডেক্স যোগ = **ফিজিক্যাল**; কলাম যোগ = **লজিক্যাল**; স্কিমা না ছুঁয়ে ফাইলের বিন্যাস বদলানো = চিরন্তন **ফিজিক্যাল** উদাহরণ।

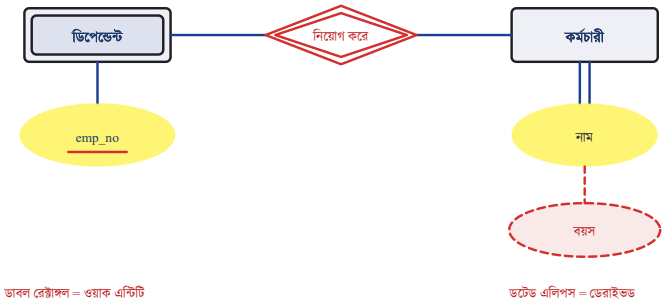
সমাধানকৃত উদাহরণ ✍️ বদল কোনটি:
"অ্যাডমিন *dept*-এ হ্যাশ ইনডেক্স দিয়ে টেবিল সাজালেন।" → **ফিজিক্যাল**: কোয়েরির অর্থ একই। "Student-এ *nationality* কলাম যোগ হলো।" → **লজিক্যাল**।

IT link: এক ইঞ্জিন থেকে নেওয়া ডাম্প অন্য ইঞ্জিনে চালানো যায় এর জন্যই। বড় টেবিলে `ALTER TABLE … ADD COLUMN` করা সব প্রোগ্রাম নতুন করে লেখার চেয়ে সস্তা। **ভিউ** হলো বাস্তবায়িত এক্সটার্নাল স্তর।

৪ চেনের নোটেশনে এন্টিটি-রিলেশনশিপ মডেল

সংজ্ঞা: **E-R মডেল** (পিটার চেন, **১৯৭৬**) একটি উচ্চ-স্তরের **কনসেপচুয়াল** মডেল, যা দিয়ে টেবিল বানানোর আগে ডেটাবেস ডিজাইন করা হয়। এর তিনটি মূল উপাদান — **এন্টিটি**, **অ্যাট্রিবিউট**, **সম্পর্ক** — এবং প্রতিটির নির্দিষ্ট প্রতীক আছে।

> চেন, ১৯৭৬



ওয়াক এন্টিটি, আইডেন্টিফাইং সম্পর্ক, কী, ডেরাইভড

উপাদান	প্রতীক	উদাহরণ
স্ট্রং এন্টিটি	আয়তক্ষেত্র	কর্মচারী
ওয়াক এন্টিটি	**ডাবল রেক্টাঙ্গল**	ডিপেন্ডেন্ট
অ্যাট্রিবিউট	ডিম্বাকৃতি	নাম, id
কী অ্যাট্রিবিউট	**আন্ডারলাইন**	id
ডেরাইভড	ডটেড এলিপস	বয়স
মাল্টিভ্যালুড	ডাবল এলিপস	ফোন নম্বর
কম্পোজিট	ভেতরে ছোট ডিম্বসহ ডিম্ব	ঠিকানা
পার্টিয়াল কী	ওয়াক এন্টিটির নিজস্ব আংশিক কী	ডিপেন্ডেন্টের ধরন
অস্তিত্ব-নির্ভরশীল	মালিক মুছলে মুছে যায়	ডিপেন্ডেন্ট
সম্পর্ক	হীরক	Enrolls
আইডেন্টিফাইং	ডাবল হীরক	অধিপতি
কার্ডিনালিটি	১, N বা M	১:N, M:N
টোটাল পার্টিসিপেশন	ডাবল রেখা	প্রত্যেক কর্মচারীর বিভাগ আছে

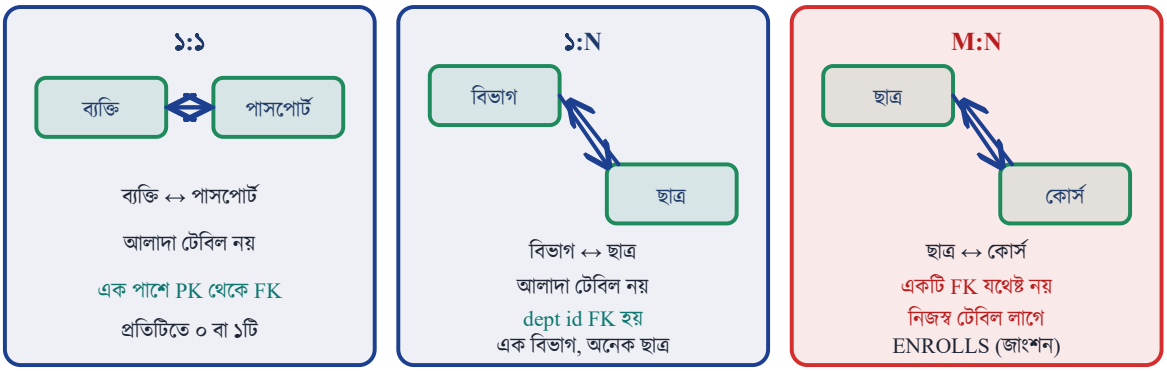
পরীক্ষার ফাঁদ: **ওয়াক এন্টিটি সেট আঁকা হয় ডাবল রেক্টাঙ্গল** — হীরক নয় (সেটি সম্পর্ক), ডিম্বাকৃতি নয় (সেটি অ্যাট্রিবিউট), ডটেড ডিম্বও নয় (সেটি *ডেরাইভড*)। এর আইডেন্টিফাইং সম্পর্ক **ডাবল হীরক**। ওয়াক এন্টিটির **নিজস্ব কী থাকে না**: চিহ্নিত হয় *মালিকের কী + নিজের পার্টিয়াল কী* দিয়ে।

মনে রাখার কৌশল: **"আয়তক্ষেত্র এন্টিটি, ডিম্ব অ্যাট্রিবিউট, হীরক সম্পর্ক, আন্ডারলাইন = কী।"** প্রতিটি ব্যতিক্রমে একটি অক্ষর: **ডাবল** রেক্টাঙ্গল = ওয়াক, **ডটেড** এলিপস = ডেরাইভড, **ডাবল** এলিপস = মাল্টিভ্যালুড, **ডাবল** হীরক = আইডেন্টিফাইং। **কম্পোজিট** = ভেতরে ছোট ডিম্বসহ ডিম্ব (*ঠিকানা*)।

সমাধানকৃত উদাহরণ ✍️ E-R ডায়াগ্রাম পড়া: "প্রতিটি **বিভাগ** আয়তক্ষেত্রে একটি আন্ডারলাইন করা **নাম** আছে এবং একটি ডটেড এলিপস **staff_cnt**। একটি **ডাবল রেক্টাঙ্গল অ্যাসাইনমেন্ট** PROJECT-কে যুক্ত করে।" — *নাম* স্ট্রং এন্টিটির কী অ্যাট্রিবিউট; *staff_cnt* **ডেরাইভড**; **অ্যাসাইনমেন্ট** একটি **ওয়াক এন্টিটি**, কোন প্রজেক্ট ও কোন কর্মচারীর জন্য তা জানা ছাড়া অর্থহীন।

৫ কার্ডিনালিটি: ১:১, ১:N ও M:N

সংজ্ঞা: **কার্ডিনালিটি** জানায় একটি এন্টিটির একটি উদাহরণের সঙ্গে অপরটির সর্বোচ্চ কতগুলো উদাহরণ যুক্ত হতে পারে। আকার মাত্র তিনটি — **১:১**, **১:N**, **M:N** — এবং সেই আকারই ঠিক করে **ডিজাইনে কতগুলো রিলেশন লাগবে**।



১:N-এ আলাদা টেবিল লাগে না; M:N-এ সবসময় একটি লাগে

শুধু আকার দেখেই টেবিলের সংখ্যা বলা যায়

কার্ডিনালিটি	অর্থ	উদাহরণ	রিলেশনে ম্যাপিং
১:১	একজন A, সর্বোচ্চ একজন B	ব্যক্তি ↔ পাসপোর্ট	নতুন টেবিল নয়: "এক" পাশের **PK অন্য টেবিলে FK**
১:N	একজন A, অনেকজন B	বিভাগ ↔ ছাত্র	নতুন টেবিল নয়: **"এক" পাশের PK "অনেক" পাশে FK**
M:N	অনেকজন A, অনেকজন B	ছাত্র ↔ কোর্স	**নিজস্ব রিলেশন**, দুই পাশেই FK
তথ্যসহ ১:N	সম্পর্কটিও ডেটা রাখে (*since*, *grade*)	ছাত্র ↔ Enrolls	১:N-এও জাংশন টেবিল লাগে
বাধ্যতামূলক ১:N	প্রতিটি ছাত্রের অবশ্যই একটি বিভাগ	বিভাগ ↔ ছাত্র	dept_id NOT NULL রাখতে হয়
ঐচ্ছিক ১:N	ছাত্রের বিভাগ নাও থাকতে পারে	ছাত্র ↔ উপদেষ্টা	FK রাখা যায়, NULL-ও থাকতে পারে

পরীক্ষার ফাঁদ: **১:N-এ foreign key থাকে N পাশে**; "এক" পাশে রাখলে একটি ঘরে কয়েকটি id লাগবে, যা ১NF ভাঙে। **M:N কোনো একক foreign key দিয়ে লেখা যায় না**: একজন ছাত্রের অনেক কোর্স *এবং* একটি কোর্সের অনেক ছাত্র, তাই কোনো টেবিলেই অপরের তালিকা রাখা যায় না। **শুধু M:N-ই নতুন রিলেশন বাধ্য করে।**

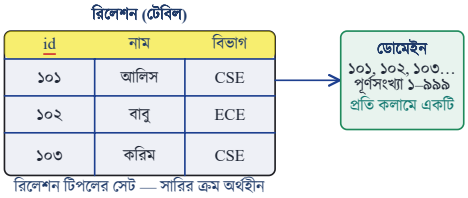
সমাধানকৃত উদাহরণ ✍️ M:N-এ টেবিল কেন লাগে: `Student`-এ রাখো: `courses` কলামে `{CSE101, MAT201}` জমতে হতো — এটি **মাল্টিভ্যালুড ঘর**, যা ১NF নিষিদ্ধ করে। `Course`-এ রাখলে একই সমস্যা। তাই **ENROLLS(student_id, course_code)**, দুই পাশেই FK; জোড়াটি সাধারণত কম্পোজিট কীও।

১:১-এ কোন পাশে রাখব: যে পাশে **সংখ্যা না থাকতে পারে** তার দিকে FK রাখো: কর্মচারী টেবিলে `passport_no` গেলে পাসপোর্টনহীন কর্মচারীর জায়গায় NULL থাকে। **১:N-এ পাশ বাছাই নয়** — কী সবসময় N পাশে যায়।

৬ রিলেশনাল মডেল

সংজ্ঞা: **রিলেশনাল মডেল** (ই. এফ. কড, ১৯৭০) ডেটাকে **রিলেশন** হিসেবে দেখায় — নাম দেওয়া অ্যাট্রিবিউটের উপর টিপলের গাণিতিক সেট। রিলেশনকে টেবিল হিসেবে দেখানো হয়: টিপল = সারি, অ্যাট্রিবিউট = কলাম। দুটি বৈশিষ্ট্য প্রতিটি উত্তরে লাগে: রিলেশনে **ভিন্ন টিপল থাকে না** এবং **কোনো ক্রম থাকে না**।

> কড, ১৯৭০



সারি ঢোকে, সারি বেরোয়; কলাম বহন করে ডোমেইন

আনুষ্ঠানিক পরিভাষা	সাধারণ নাম	অর্থ
রিলেশন	টেবিল	একই অ্যাট্রিবিউটের টিপলের সেট
টিপল	সারি / রেকর্ড	একটি তথ্য: `(১০১,"আলিস","CSE")`
অ্যাট্রিবিউট	কলাম / ক্ষেত্র	নাম দেওয়া বৈশিষ্ট্য: *নাম*, *বিভাগ*
ডোমেইন	অনুমোদিত মানের সেট	*id*-এর জন্য ১–৯৯৯
ডিগ্রি	আরিটি, কলাম সংখ্যা	Student-এ ডিগ্রি **৩**
কার্ডিনালিটি	সারি সংখ্যা	Student-এ ৩টি টিপল
রিলেশন স্কিমা	টেবিলের সংজ্ঞা	`Student(id, name, dept)`
রিলেশন ইনস্ট্যান্স	বর্তমান বিষয়বস্তু	এই মুহূর্তের সারিগুলো
কী	টিপল শনাক্তকারী অ্যাট্রিবিউট	আন্ডারলাইন করা *id*

আনুষ্ঠানিক ধর্ম: ঘরে শুধু **অ্যাটমিক** মান · টিপল **আলাদা** · টিপল ও অ্যাট্রিবিউটের **ক্রম অর্থহীন** · প্রতিটি টিপলে একই অ্যাট্রিবিউট, একই ডোমেইন।

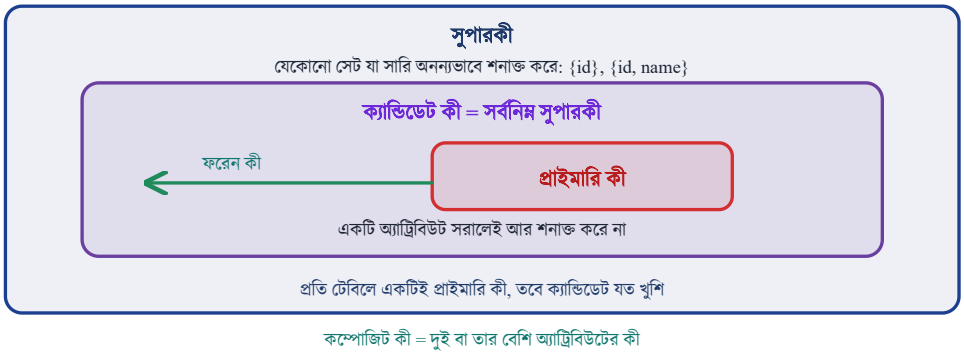
মনে রাখার কৌশল: "**টেবিল–সারি–কলাম–সেট।**" "ডিগ্রি সারি না কলাম?" → **কলাম**। "কার্ডিনালিটি সারি না কলাম?" → **সারি**। পরীক্ষকারী এই দুটি অদল-বদল করেন।

পরীক্ষার ফাঁদ: পর্দায় টেবিল **সাজানো থাকতে পারে**, কিন্তু রিলেশন একটি সেট, তাই **সারির ক্রম ডেটার অংশ নয়**। সারি যোগ করলে **ডিগ্রি বদলায় না**। হুবহু একই দুটি সারি **চলবে না**।

সমাধানকৃত উদাহরণ ✍️ একটি টেবিল আনুষ্ঠানিকভাবে বর্ণনা: `Student(id, name, dept)`-এ `(১০১,"আলিস","CSE")`, `(১০২,"বাবু","ECE")`, `(১০৩,"করিম","CSE")`। রিলেশনের নাম **Student**; ডিগ্রি **৩** (তিনটি অ্যাট্রিবিউট); কার্ডিনালিটি **৩** (তিনটি টিপল); *id*-এর ডোমেইন পূর্ণসংখ্যা ১–৯৯৯; *id* আন্ডারলাইন, তাই কী অ্যাট্রিবিউট। চতুর্থ ছাত্র যোগে কার্ডিনালিটি ৪ হয়, ডিগ্রি ৩-ই থাকে।

৭ কী: সুপার, ক্যান্ডিডেট, প্রাইমারি, ফরেন, কম্পোজিট

সংজ্ঞা: **সুপারকী** হলো টিপলকে অনন্যভাবে শনাক্ত করে এমন যেকোনো অ্যাট্রিবিউটের সেট। **ক্যান্ডিডেট কী** হলো এমন সুপারকী যা **সর্বনিম্ন** — একটি অ্যাট্রিবিউট সরালেই আর শনাক্ত করে না। **প্রাইমারি কী** হলো সেই একটি ক্যান্ডিডেট কী যা আনুষ্ঠানিক শনাক্তকারী মনে করা হয়। **ফরেন কী** অন্য রিলেশনের ক্যান্ডিডেট কী-র দিকে নির্দেশ করে। **কম্পোজিট কী** দুই বা তার বেশি অ্যাট্রিবিউট দিয়ে তৈরি।



প্রতিটি কী-ই সুপারকী; সব সুপারকী কিন্তু কী নয়

কী	এটা কী	নিয়ম	উদাহরণ
সুপারকী	টিপল শনাক্ত করে এমন সেট	প্রয়োজনের চেয়ে বড় হতে পারে	{id, name}
ক্যান্ডিডেট কী	**সর্বনিম্ন** সুপারকী	কিছু সরানো যায় না	{id}
প্রাইমারি কী	বেছে নেওয়া ক্যান্ডিডেট কী	**টেবিলে একটি; NULL নয়**	PRIMARY KEY (id)
ফরেন কী	অন্য টেবিলের কী-র দিকে নির্দেশ	রক্ষা করে **রেফারেনশিয়াল ইন্টেগ্রিটি**	`dept_id REFERENCES Dept(id)`
কম্পোজিট কী	২ বা তার বেশি অ্যাট্রিবিউট	প্রাইমারি কী কম্পোজিট হতে পারে	(StudentID, CourseID)
অল্টারনেট কী	বাছাই না করা ক্যান্ডিডেট কী	তাও UNIQUE থাকে	{passport_no}

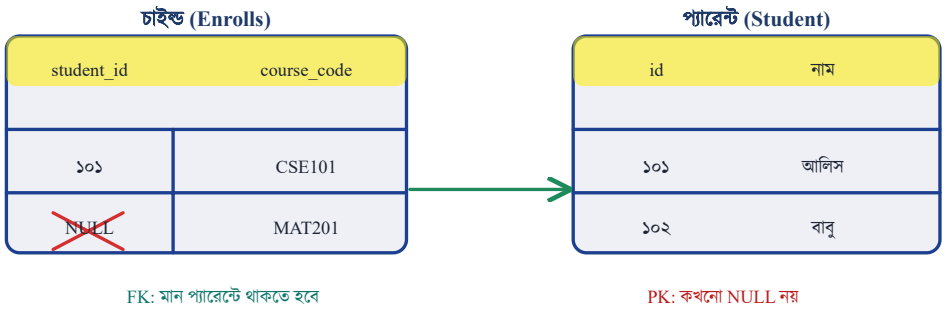
মনে রাখার কৌশল: **"প্রতিটি ক্যান্ডিডেট কী একটি সুপারকী; প্রতিটি প্রাইমারি কী একটি ক্যান্ডিডেট কী; ফরেন কী অন্যের প্রাইমারি কী।"** সাঁটানোটি একদিকে: সুপারকী ⊇ ক্যান্ডিডেট কী ⊇ প্রাইমারি কী।

পরীক্ষার ফাঁদ: একটি টেবিলে **ঠিক একটিই** প্রাইমারি কী, তবে অনেক ক্যান্ডিডেট কী ও সুপারকী থাকতে পারে। প্রাইমারি কী **কম্পোজিট** হতে পারে — এক কলাম হওয়া বাধ্যতামূলক নয়। ফরেন কী কলামে **NULL থাকতে পারে**; তার NULL-নয় মানগুলোই মিলতে হবে।

সমাধানকৃত উদাহরণ ✍️ একটি টেবিলের কী শ্রেণিবিভাগ: `Person(pid, passport_no, phone)`। {pid} অনন্য → সুপারকী; *pid* সরালে ব্যর্থ → সর্বনিম্ন → **ক্যান্ডিডেট কী**; বেছে নেওয়া → **প্রাইমারি কী**। {passport_no}-ও সর্বনিম্ন → **অল্টারনেট কী**। {pid, phone} অনন্য, কিন্তু *phone* সরানো যায় বলে সেটি **ক্যান্ডিডেট কী নয়** — শুধু সুপারকী।

৮ কনস্ট্রেইন্ট ও দুটি ইন্টেগ্রিটি নিয়ম

সংজ্ঞা: **কনস্ট্রেইন্ট** হলো কোনো কলাম বা টেবিলে ঘোষিত নিয়ম, যা লেখার যেকোনো উপায়েই ডেটাকে সঠিক রাখে। SQL-এ কনস্ট্রেইন্টের ছয় রূপ আছে, আর সেগুলো দুটি নাম দেওয়া বৈশিষ্ট্য রক্ষা করে: **এনটিটি ইন্টেগ্রিটি** ও **রেফারেনশিয়াল ইন্টেগ্রিটি**।



দুটি ইন্টেগ্রিটি নিয়ম

FK-র তির কখনো ফাঁকায় পড়তে পারে না

কনস্ট্রেইন্ট	যা নিষিদ্ধ করে	SQL
NOT NULL	কলামে NULL মান	`name VARCHAR(50) NOT NULL`
UNIQUE	দুই সারিতে একই মান (NULL ছাড়া)	`email VARCHAR(100) UNIQUE`
PRIMARY KEY	NULL ও ডুপ্লিকেট — উপরের দুটিই ধারণ করে	`id INT PRIMARY KEY`
FOREIGN KEY	যে মান রেফারেন্স করা টেবিলের কী-র সঙ্গে মেলে না	`dept_id INT REFERENCES Dept(id)`
CHECK	বলা শর্তে মান না পূরা করা	`CHECK (Age >= 18)`
DEFAULT	কিছু নয় — মান না দিলে একটি দেয়	`status VARCHAR(20) DEFAULT 'active'`

কনস্ট্রেইন্ট যে দুটি নিয়ম রক্ষা করে: **এনটিটি ইন্টেগ্রিটি** — **প্রাইমারি কী কখনো NULL হয় না** এবং প্রতিটি সারিতে অনন্য। যে রিলেশনকে ঠিকানা বলা যায়, তার সারির পরিচয় থাকতে হবে; না হলে অন্য টেবিল তাকে নির্দেশ করতে পারে না। রক্ষা করে **PRIMARY KEY**। **রেফারেনশিয়াল ইন্টেগ্রিটি** — **ফরেন কী-র প্রতিটি NULL-নয় মান রেফারেন্স করা টেবিলের কোনো কী-র সঙ্গে মেলে**। কোনো অসংযুক্ত সারি থাকবে না। রক্ষা করে **FOREIGN KEY**; প্যারেন্ট মুছলে বেছে নিতে হয় `CASCADE`, `SET NULL`, `SET DEFAULT` বা `RESTRICT`।

```
CREATE TABLE Enrolls ( student_id  INT      NOT NULL REFERENCES Student(id),
                       course_code CHAR(7)  NOT NULL REFERENCES Course(code),
                       grade       CHAR(2)  CHECK (grade IN ('A','B','C','F')),
                       enrolled_on DATE     DEFAULT CURRENT_DATE );
```

পরীক্ষার ফাঁদ ✍️ দুটি: **১.** ফরেন কী আছে **রেফারেনশিয়াল ইন্টেগ্রিটি** বলবার জন্য — এনটিটি ইন্টেগ্রিটি, এনক্রিপশন বা দ্রুত সাজানো নয়; এনটিটি ইন্টেগ্রিটি প্রাইমারি কী-এর কাজ। FK কলামে **NULL থাকতে পারে**। **২. DROP** টেবিলের **কাঠামো ও ডেটা দুটোই** মুছে দেয়; **DELETE** সারি মোছে, **TRUNCATE** সব সারি মোছে, কিন্তু **দুটোই কাঠামো রাখে**; **ALTER** কাঠামো বদলায়, ডেটা রাখে।

৯ ট্রিগার, এবং E-R বনাম রিলেশনাল মডেল

সংজ্ঞা: **ট্রিগার** হলো সংরক্ষিত যৌক্তিকতা, যা কোনো টেবিলে কোনো **ঘটনা ঘটলে DBMS নিজে থেকেই** চালায় — `INSERT`, `UPDATE` বা `DELETE`-এর আগে, পরে, বা বদলে। অ্যাপ্লিকেশন কখনো ট্রিগার ডাকে না; চালায় *ইঞ্জিন*, তাই কোনো ক্লায়েন্ট তাকে বাঁচাতে পারে না।

সময়	কখন চলে
BEFORE	সারি লেখার আগে; NEW বদলাতে পারে
AFTER	সারি লেখার পরে; OLD ও NEW দুটোই পাওয়া যায়
FOR EACH ROW	প্রতিটি প্রভাবিত সারিতে একবার
INSTEAD OF	**ভিউ**-র উপর, ভিউকে আপডেটেবল করতে

পরীক্ষার ফাঁদ: ট্রিগার **স্টোরড প্রসিডিউর নয়**: কেউ নাম ধরে ডাকে না, এটি **ঘটনা**-তে চলে। কনস্ট্রেইন্টের বিকল্পও নয় — একই কাজের জন্য CHECK সস্তা ও পরিষ্কার। **ফরেন কী বাধ্যতামূলকতা কনস্ট্রেইন্ট দিয়ে হয়, ট্রিগার দিয়ে নয়।**

```
CREATE TRIGGER trg_student_count
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
  UPDATE Department
  SET StudentCount = StudentCount + 1
  WHERE DeptID = NEW.DeptID;
END;
```

সমাধানকৃত উদাহরণ ✍️ তিনটি ব্যবহার: **১. সংখ্যা ধরে রাখা:** উপরের ট্রিগার প্রতিটি insert-এর পরে ওই বিভাগের মোট ছাত্র এক বাড়ায়। **২. অডিট:** `salary` বদলালে পুরনো ও নতুন মান `Salary_Log`-এ কপি হয়। **৩. যাচাই:** insert-এর আগে অতীতের তারিখের বুকিং ঠেকানো — তবে এখানে CHECK ভালো।

বিষয়	E-R মডেল (কনসেপচুয়াল)	রিলেশনাল মডেল
ধাপ	**ডিজাইন** — টেবিল বানানোর আগে	**বাস্তবায়ন** — টেবিলগুলোই
ডেটা হিসেবে	এন্টিটি, অ্যাট্রিবিউট, সম্পর্ক	রিলেশন, টিপল, অ্যাট্রিবিউট
অ্যাবস্ট্রাকশন	উচ্চ — যৌক্তিক, বাস্তবায়ন-মুক্ত	নিম্ন — স্টোরেজের কাছাকাছি
ব্যবহারকারী	ডিজাইনার ও বিশ্লেষক	প্রোগ্রামার ও DBA
কোন স্তরে	ত্রি-স্তরের **কনসেপচুয়াল** স্তর	বাস্তবায়নের **এক্সটার্নাল ও ইন্টার্নাল** স্তর

অধ্যায়ের বাকি অংশ কোথায়: **রিলেশনাল অ্যালজেব্রা ও SQL** (সিলেকশন σ, প্রজেকশন π, জয়ন; SELECT, WHERE, GROUP BY, HAVING) → কোয়েরি ভাষার লেসন। **ফাংশনাল ডিপেন্ডেন্সি, ১NF–BCNF, নরমালাইজেশন** → নরমালাইজেশন লেসন। **ফাইল সংগঠন, ইনডেক্স, B+ ট্রি, হ্যাশ টেবিল** → ইনডেক্স লেসন। **ACID, ট্রানজ্যাকশন, কনকারেন্সি, রিকভারি** → ট্রানজ্যাকশন লেসন।

রাতের পরীক্ষার পাতা

তথ্যপাতা:

ডেটাবেস সিস্টেম = DB + DBMS + অ্যাপ + ব্যবহারকারী

DB = ডেটা; DBMS = পরিচালনার সফটওয়্যার

স্কিমা = ব্লুপ্রিন্ট; ইনস্ট্যান্স = এখনকার ডেটা

এক্সটার্নাল → কনসেপচুয়াল → ইন্টার্নাল

এক্সটার্নাল = ভিউ · ইন্টার্নাল = ফিজিক্যাল

ইন্ডিপেন্ডেন্স: লজিক্যাল ও ফিজিক্যাল — দুটি

চেন: এন্টিটি = আয়তক্ষেত্র

অ্যাট্রিবিউট = ডিম্ব · কী = আন্ডারলাইন

সম্পর্ক = হীরক · আইডেন্টিফাইং = ডাবল হীরক

ওয়াক এন্টিটি = **ডাবল রেক্টাঙ্গল** · ডেরাইভড = ডটেড

১:১ ও ১:N-এ আলাদা টেবিল নেই

M:N-এ সবসময় নিজস্ব জাংশন রিলেশন লাগে

ডিগ্রি = কলাম সংখ্যা · কার্ডিনালিটি = সারি সংখ্যা

রিলেশন = টিপলের সেট; নকল নেই, ক্রম নেই

সুপারকী ⊇ ক্যান্ডিডেট কী ⊇ প্রাইমারি কী

প্রতি টেবিলে একটিই প্রাইমারি কী; NULL নয়

ফরেন কী → রেফারেনশিয়াল ইন্টেগ্রিটি

প্রাইমারি কী → এনটিটি ইন্টেগ্রিটি

ট্রিগার = ঘটনায় স্বয়ংক্রিয় যৌক্তিকতা

১:N-এ চাইলে জাংশন টেবিলও লাগে

DROP কাঠামো মোছে; DELETE/TRUNCATE মোছে না

MCQ-র জন্য সেরা ১২টি এক-লাইনের কথা

1. ডেটাবেস সিস্টেম ≠ ডেটাবেস ≠ DBMS
2. স্কিমা = ব্লুপ্রিন্ট, ইনস্ট্যান্স = এখনকার ডেটা
3. ANSI/SPARC: এক্সটার্নাল, কনসেপচুয়াল, ইন্টার্নাল
4. ইন্ডিপেন্ডেন্সের ধরন ঠিক **দুটি**
5. ইনডেক্স যোগ = **ফিজিক্যাল** ইন্ডিপেন্ডেন্স
6. চেনে এন্টিটি আয়তক্ষেত্র, কী আন্ডারলাইন ডিম্ব
7. ওয়াক এন্টিটি সেট = **ডাবল রেক্টাঙ্গল**
8. ১:N-এ FK থাকে **N পাশে**
9. শুধু M:N-ই জাংশন টেবিল বাধ্য করে
10. ডিগ্রি = কলাম, কার্ডিনালিটি = সারি
11. FK বাধ্যতামূলক করে **রেফারেনশিয়াল** ইন্টেগ্রিটি
12. **DROP** কাঠামো মোছে; TRUNCATE মোছে না

সবচেয়ে বড় ফাঁদগুলো: ওয়াক এন্টিটি হীরক বা ডিম্ব নয় · একটি প্রাইমারি কী, অনেক ক্যান্ডিডেট কী · প্রাইমারি কী কম্পোজিট হতে পারে · FK কলামে NULL থাকতে পারে · ইন্ডিপেন্ডেন্স দুই ধরনের · ডিগ্রি ও কার্ডিনালিটি অদল-বদল · DROP বনাম DELETE বনাম TRUNCATE।

সব কৌশল একসাথে: আয়তক্ষেত্র এন্টিটি, ডিম্ব অ্যাট্রিবিউট, হীরক সম্পর্ক, আন্ডারলাইন কী · ডাবল রেক্টাঙ্গল ওয়াক, ডটেড এলিপস ডেরাইভড · এক বিভাগ, অনেক ছাত্র ⇒ অনেক পাশে FK · M:N-এ নিজস্ব টেবিল · সুপারকী ⊇ ক্যান্ডিডেট ⊇ প্রাইমারি · এ-নটিটি = PK-তে NULL নেই, রে-ফারেনশিয়াল = FK মিলতে হবে।



তিন স্তর, এক অর্থ

সমাধানকৃত উদাহরণ ✍️ এক ডিজাইন, তিনটি রিলেশন: "বিভাগ ১:N ছাত্র, ছাত্র M:N কোর্স।" → `Student`-এ **dept_id** FK (N পাশে); আলাদা `Enrolls(student_id, course_code)` তৈরি হয়, কারণ M:N-এ একটি FK চলে না। "কয়টি রিলেশন?" → ৩।

✓ অনুশীলনী MCQ (পরীক্ষার ধরনে, ১০ নম্বর)

১. ডেটাবেস সিস্টেমের অংশ:
ক) শুধু DBMS খ) শুধু ডেটাবেস
গ) DB + DBMS + অ্যাপ + ব্যবহারকারী ঘ) শুধু ডেটা ফাইল

২. স্কিমা ও ইনস্ট্যান্স:
ক) স্কিমা = কাঠামো খ) স্কিমা = এখনকার ডেটা
গ) দুটোই ফাইলে ঘ) ইনস্ট্যান্স = ব্লুপ্রিন্ট

৩. কোড না বদলে B+ ট্রি ইনডেক্স যোগ করা:
ক) লজিক্যাল ইন্ডিপেন্ডেন্স খ) ফিজিক্যাল ইন্ডিপেন্ডেন্স
গ) এক্সটার্নাল স্তর ঘ) স্কিমা পরিবর্তন

৪. চেনের নোটেশনে ওয়াক এন্টিটি সেট আঁকা হয়:
ক) হীরক খ) ডিম্বাকৃতি
গ) ডটেড ডিম্ব ঘ) ডাবল রেক্টাঙ্গল

৫. বিভাগ ১:N কর্মচারী কীভাবে ম্যাপ হয়:
ক) বিভাগের PK কর্মচারীতে FK খ) কর্মচারীর PK বিভাগে FK
গ) দুটো এক টেবিলে ঘ) তৃতীয় টেবিল

৬. ছাত্র ↔ কোর্স M:N-এ নিজস্ব রিলেশন লাগে কারণ:
ক) অনেক অ্যাট্রিবিউট খ) একটি FK যথেষ্ট নয়
গ) লিঙ্ক দুই টেবিলে থাকে না ঘ) দুটি ইনডেক্স লাগে

৭. ফরেন কী মূলত বাধ্যতামূলক করে:
ক) এনটিটি ইন্টেগ্রিটি খ) ডেটা এনক্রিপশন
গ) রেফারেনশিয়াল ইন্টেগ্রিটি ঘ) দ্রুত সাজানো

৮. কোনটি কম্পোজিট কী?
ক) id খ) (StudentID, CourseID)
গ) name ঘ) পুরো সারি

৯. ট্রিগার প্রসিডিউর থেকে আলাদা কারণ:
ক) Java লাগে খ) ডেটা বদলাতে পারে না
গ) অ্যাপ ডাকে ঘ) DBMS-ই চালায়

১০. কোন কমান্ড কাঠামো ও ডেটা দুটোই মোছে?
ক) DELETE খ) TRUNCATE
গ) ALTER ঘ) DROP

উত্তরমালা → ১-গ · ২-ক · ৩-খ · ৪-ঘ · ৫-ক · ৬-খ · ৭-গ · ৮-খ · ৯-ঘ · ১০-ঘ

লিখিত প্রশ্ন: ডেটাবেস সিস্টেমের সংজ্ঞা দাও এবং স্কিমা ও ইনস্ট্যান্সের পার্থক্য বলো। ত্রি-স্তরের আর্কিটেকচার ও দুটি ধরনের ডেটা ইন্ডিপেন্ডেন্স ব্যাখ্যা করো। চেনের E-R নোটেশনসহ ওয়াক এন্টিটি সেটসহ চিত্র আঁকো এবং ১:১, ১:N ও M:N কীভাবে রিলেশনে ম্যাপ হয় দেখাও। সবশেষে বলো কনস্ট্রেইন্ট ও ট্রিগার কী কাজে লাগে। ১০ নম্বর

১ **সংজ্ঞা (১)** — ডেটাবেস = ডেটা; DBMS = পরিচালনার সফটওয়্যার; ডেটাবেস সিস্টেম = ডেটাবেস + DBMS + অ্যাপ্লিকেশন + ব্যবহারকারী + হার্ডওয়্যার। **স্কিমা** = ব্লুপ্রিন্ট, ডেটা ডিকশনারিতে থাকে; **ইনস্ট্যান্স** = এই মুহূর্তের সারি। ১

২ **তিন স্তর (২)** — **এক্সটার্নাল** (ব্যবহারকারীর ভিউ, প্রতি অ্যাপে একটি), **কনসেপচুয়াল** (সব ডেটার যৌক্তিক কাঠামো: টেবিল, রিলেশন, কনস্ট্রেইন্ট), **ইন্টার্নাল** (ফিজিক্যাল স্টোরেজ: ফাইল, পেজ, ব্লক, ইনডেক্স)। মান: ANSI/SPARC। ২

৩ **ডেটা ইন্ডিপেন্ডেন্স (২)** — **লজিক্যাল**: কনসেপচুয়াল স্কিমা বদলাও, এক্সটার্নাল ভিউ ও প্রোগ্রাম অপরিবর্তিত। **ফিজিক্যাল**: ইন্টার্নাল স্কিমা (ইনডেক্স, পেজ বিন্যাস) বদলাও, কনসেপচুয়াল স্কিমা ও প্রোগ্রাম অপরিবর্তিত। ২

৪ **E-R নোটেশন (২)** — এন্টিটি = আয়তক্ষেত্র (ওয়াক = **ডাবল রেক্টাঙ্গল**), অ্যাট্রিবিউট = ডিম্ব (কী **আন্ডারলাইন**, ডেরাইভড **ডটেড**), সম্পর্ক = **হীরক** (আইডেন্টিফাইং = ডাবল হীরক), কার্ডিনালিটি ১/N/M। ২

৫ **ম্যাপিং (২)** — **১:১** ও **১:N**: নতুন রিলেশন নয়; "এক" পাশের প্রাইমারি কী অন্য / "অনেক" পাশে ফরেন কী। **M:N**: একটি FK চলে না, তাই জাংশন রিলেশন। ২

৬ **কনস্ট্রেইন্ট ও ট্রিগার (১)** — ঘোষিত নিয়ম (NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK, DEFAULT) **এনটিটি ইন্টেগ্রিটি** (PK কখনো NULL নয়) ও **রেফারেনশিয়াল ইন্টেগ্রিটি** (FK মিলতে হবে) রক্ষা করে; ট্রিগার হলো ঘটনায় স্বয়ংক্রিয়ভাবে চলা সংরক্ষিত যৌক্তিকতা। ১

পরীক্ষার আগে ✓

- ✓ ডেটাবেস, DBMS, সিস্টেম আলাদা করতে পারি
- ✓ তিন স্তর ক্রমে বলতে পারি
- ✓ ইন্ডিপেন্ডেন্সের দুই ধরনের নাম বলতে পারি
- ✓ চেনের সব প্রতীক মুখস্থে আঁকতে পারি
- ✓ M:N-এ টেবিল লাগার কারণ বলতে পারি
- ✓ ছয় কনস্ট্রেইন্ট, DROP বনাম TRUNCATE বলতে পারি